

Competitive Programming

Section 32 Editorial: Combinatorics Under Modulo

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	Factorial helper	4
3	A. Choose Queries	5
3.1	Restatement	5
3.2	Algorithm	5
3.3	Correctness proof	5
3.4	Complexity	5
3.5	Full C++ solution	5
3.6	Full Python solution	6
4	B. Rearrange Letters	7
4.1	Restatement	7
4.2	Observations	7
4.3	Formula	7
4.4	Correctness proof	7
4.5	Full C++ solution	7
4.6	Full Python solution	8
5	C. Distribute Candies	9
5.1	Restatement	9
5.2	Stars-and-bars derivation	9
5.3	Correctness proof	9
5.4	Full C++ solution	9
5.5	Full Python solution	10
6	Testing discussion	11
7	What to carry forward	12

1 Editorial overview

All three problems use factorials and modular inverses under the prime modulus $1,000,000,007$.

Problem	Formula
A. Choose Queries	$C(n, k)$
B. Rearrange Letters	$n! / \text{product}(\text{count}!)$
C. Distribute Candies	$C(n+s-1, n-1)$

2 Factorial helper

Precompute:

```
fact[i] = i!  
invfact[i] = inverse(i!)
```

Then:

```
C(n,k) = fact[n] * invfact[k] * invfact[n-k] mod MOD
```

3 A. Choose Queries

3.1 Restatement

Answer many binomial coefficient queries modulo MOD.

3.2 Algorithm

Read all queries, find the largest n , precompute factorials and inverse factorials up to that limit, then answer each query in $O(1)$.

3.3 Correctness proof

$\text{fact}[n]$ stores $n!$. Since MOD is prime and $n < \text{MOD}$, the factorials in the denominator are invertible. Multiplying by $\text{invfact}[k]$ and $\text{invfact}[n-k]$ is therefore exactly division by $k!(n-k)!$ in modular arithmetic. This matches the binomial coefficient formula. $\square$

3.4 Complexity

```
Preprocessing:  $O(\max n)$ 
Each query:    $O(1)$ 
Memory:        $O(\max n)$ 
```

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
}
```

```

    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<pair<int, int>> queries(q);
    int limit = 0;
    for (auto &[n, k] : queries) {
        cin >> n >> k;
        limit = max(limit, n);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
    for (auto [n, k] : queries) {
        cout << fact[n] * invfact[k] % MOD * invfact[n - k] % MOD << '\n';
    }
    return 0;
}

```

3.6 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

```

4 B. Rearrange Letters

4.1 Restatement

Count distinct permutations of all letters in a string that may contain repeated letters.

4.2 Observations

There are $n!$ permutations if every position were distinguishable. For a letter that appears c times, permutations among those equal copies are not distinct, so divide by $c!$.

4.3 Formula

```
answer = n! / product(count[ch]!)
```

4.4 Correctness proof

Label equal letters temporarily. There are $n!$ labeled permutations. For any fixed final string, the `count[ch]` copies of each character can be permuted among themselves without changing the visible string, giving exactly $\text{product}(\text{count}[\text{ch}]!)$ labeled permutations for the same visible arrangement. Dividing gives the number of distinct arrangements. Modular inverses implement the same division under MOD. `\square`

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
}
```

```

    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;
    int n = (int)s.size();
    vector<long long> fact(n + 1, 1);
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    vector<int> cnt(26, 0);
    for (char ch : s) cnt[ch - 'a']++;
    long long answer = fact[n];
    for (int c : cnt) answer = answer * mod_pow(fact[c], MOD - 2) % MOD;
    cout << answer << '\n';
    return 0;
}

```

4.6 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = sys.stdin.buffer.read().strip()
    if not data:
        return
    n = len(data)
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
    counts = [0] * 26
    for ch in data:
        counts[ch - 97] += 1
    answer = fact[n]
    for c in counts:
        answer = answer * pow(fact[c], MOD - 2, MOD) % MOD
    print(answer)

if __name__ == "__main__":
    main()

```

5 C. Distribute Candies

5.1 Restatement

Count nonnegative integer solutions:

$$x_1 + x_2 + \dots + x_n = s$$

5.2 Stars-and-bars derivation

Represent candies by stars and separators between children by bars. There are s stars and $n-1$ bars, so $s+n-1$ positions. Choosing the bar positions uniquely determines the distribution.

Therefore:

$$\text{answer} = C(s+n-1, n-1)$$

5.3 Correctness proof

Given a distribution, write x_1 stars, then a bar, then x_2 stars, and so on. This creates a unique sequence of s stars and $n-1$ bars. Conversely, any such sequence determines how many stars lie in each of the n gaps. This is a bijection, so the number of distributions is the number of ways to choose the bar positions: $C(s+n-1, n-1)$. $\square$

5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<pair<int, int>> queries(q);
    int limit = 0;
    for (auto &[n, s] : queries) {
        cin >> n >> s;
        limit = max(limit, n + s - 1);
    }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; i--) invfact[i - 1] = invfact[i] * i % MOD;
    for (auto [n, s] : queries) {
        int total = n + s - 1;
        int choose = n - 1;
        cout << fact[total] * invfact[choose] % MOD * invfact[total - choose] % MOD << '\n';
    }
    return 0;
}

```

5.5 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

```

6 Testing discussion

The random tests compare against Python's exact `math.comb` and exact factorial arithmetic for small cases, then reduce modulo MOD. Important cases include:

```
k = 0 and k = n
s = 0
n = 1
all letters equal
all letters distinct
```

7 What to carry forward

Most beginner combinatorics under modulo reduces to:

```
precompute factorials  
precompute inverse factorials  
translate the story into  $C(n,k)$ , duplicate permutations, or stars-and-bars
```

The next section adds inclusion-exclusion for cases where direct counting overcounts invalid objects.